BUILD / TEST / EXPLAIN

AWS

VPC Peering

Connect two VPCs.
Prove private connectivity.

EC2, custom route tables, security groups and a bastion host (SSH ProxyJump)



2 VPCs / 3 EC2 INSTANCES / 1 PRIVATE CONNECTION

AWS ARCHITECT ASSOCIATE

Name	**Harsha Vardhan**
Batch	_____
Project	**Project 01 - VPC Peering**
Region	**Asia Pacific (Sydney) / ap-southeast-2**
Completion date	**18 October 2026**

01 / PROJECT SUMMARY

What was built and what was proven

This project connects two private VPCs with a VPC peering connection and proves that an application server in each VPC can reach the other over private IPv4 addresses only. A single public bastion host in VPC A is the only internet entry point and is used for administration (laptop → bastion → App-A or App-B using SSH ProxyJump). Both application servers have no public IP address. Everything was built by hand in one AWS account and one Region (Asia Pacific (Sydney) / ap-southeast-2, Availability Zone ap-southeast-2a) using Amazon Linux 2023 on t3.micro instances.

Results at a glance

Test	Where / command	Expected	Observed	Evidence
01 Baseline (before peering)	App-A: `ping -c 4 10.20.1.10`	No replies	**4 sent, 0 received, 100% loss**	Fig. 18
02 Forward path	App-A: `ping -c 4 10.20.1.10`	4 replies, 0% loss	**4/4 replies, 0% loss**	Fig. 19
03 Reverse path	App-B: `ping -c 4 10.10.2.10`	4 replies, 0% loss	**4/4 replies, 0% loss**	Fig. 20
04 Private SSH	Laptop: `ssh ewb-app-b`	Shell on App-B via bastion	**Shell on ip-10-20-1-10**	Fig. 20
05 Isolation	EC2 instance details	No public IPv4 on App-A / App-B	**Public IPv4 = – on both**	Fig. 16, 17
Controlled failure	Remove App-B ICMP Echo rule, ping from App-A	Replies stop	**100% loss** (run twice)	Fig. 21, 22, 25, 26
Recovery	Restore the exact rule, ping again	Replies resume	**4/4 replies, 0% loss**	Fig. 23, 24

Architecture

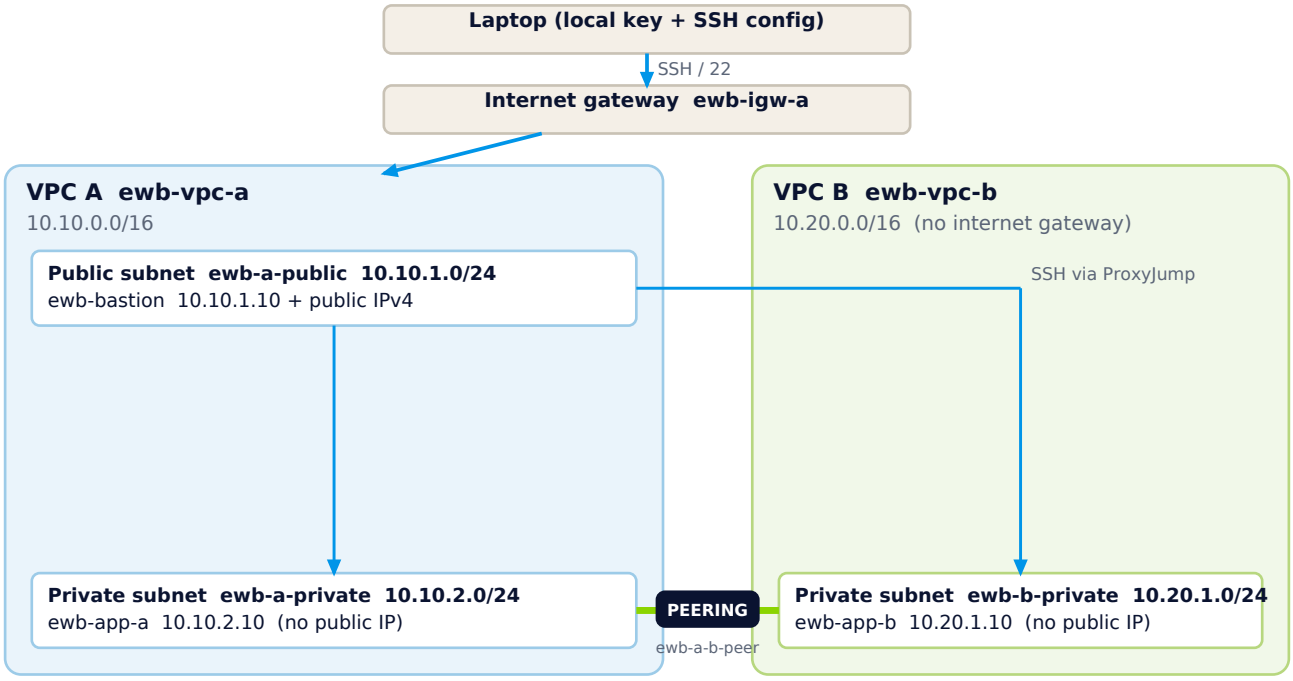


Diagram. One internet entry point (the bastion), two private workloads and a direct peering connection. The diagram is conceptual: the bastion opens a separate SSH connection to the destination. Peering does not give VPC B internet access through VPC A.

02 / ADDRESS PLAN AND INVENTORY

Address plan and resource inventory

The two VPC ranges (10.10.0.0/16 and 10.20.0.0/16) do not overlap, which is the prerequisite for peering. Each /24 subnet sits inside its own VPC range. Private IPs were fixed at launch so that the /32 security group rules match exactly.

Resource	Name tag	ID	CIDR / address
VPC A	ewb-vpc-a	vpc-04a86c6834dc99c21	10.10.0.0/16
VPC B	ewb-vpc-b	vpc-04db17c1a53e0d5d4	10.20.0.0/16
Public subnet (A)	ewb-a-public	subnet-0b7a007f14f917957	10.10.1.0/24
Private subnet (A)	ewb-a-private	subnet-029cbb7ea82f5c5b6	10.10.2.0/24
Private subnet (B)	ewb-b-private	subnet-07fcae318b98d9971	10.20.1.0/24
Internet gateway	ewb-igw-a	igw-000e7deb21fb0f126	attached to VPC A
Route table	ewb-rt-a-public	rtb-0373b24945f13e4b1	assoc. ewb-a-public
Route table	ewb-rt-a-private	rtb-0fc5b7429fc93f554	assoc. ewb-a-private
Route table	ewb-rt-b-private	rtb-0ec82061b292176b8	assoc. ewb-b-private
Security group	ewb-sg-bastion	sg-0ff752394086015a4	VPC A
Security group	ewb-sg-app-a	sg-03197dc9ad7c0d7f4	VPC A
Security group	ewb-sg-app-b	sg-00f4102508a6e5776	VPC B
Peering connection	ewb-a-b-peer	pcx-08b50e46f15a6073a	VPC A ↔ VPC B (Active)
EC2 instance	ewb-bastion	i-0f2c142ed003c399f	10.10.1.10 + public 32.236.80.143
EC2 instance	ewb-app-a	i-09abe8802cf107222	10.10.2.10 / no public IP
EC2 instance	ewb-app-b	i-0a07719a7e31f4cf5	10.20.1.10 / no public IP
Key pair	ewb-lab	RSA, .pem (kept on laptop only)	-

All resources carry the tags Project = EWB-VPC-Peering and Student = Harsha Vardhan. Instances: Amazon Linux 2023 (ami-085ce13fd231ab4ae), t3.micro, 8 GiB root volume with delete-on-termination.

03 / BUILD THE NETWORK

VPCs, subnets and internet gateway

VPC A and VPC B were created with the *VPC only* wizard so that every subnet and route is explicit. VPC A holds the public and private subnets; VPC B has a single private subnet and no internet gateway.

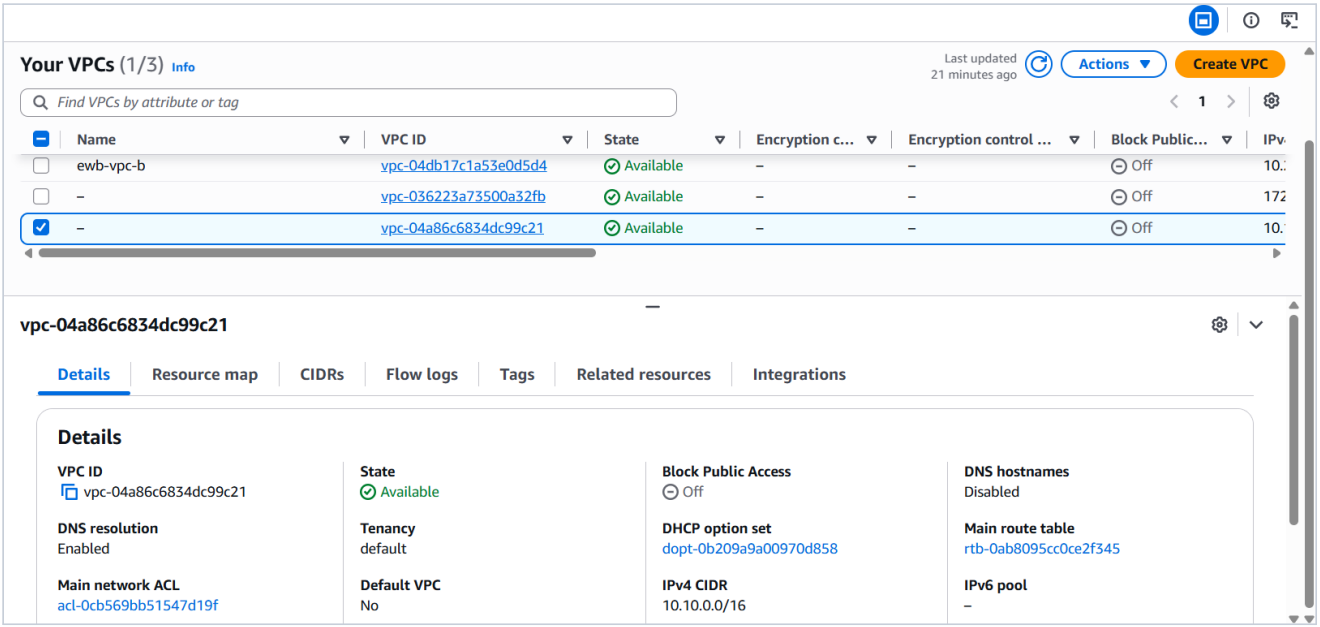


Figure 1. VPC A – ewb-vpc-a, `vpc-04a86c6834dc99c21`, IPv4 CIDR 10.10.0.0/16, tenancy default (screenshot 177, 1:42 PM).

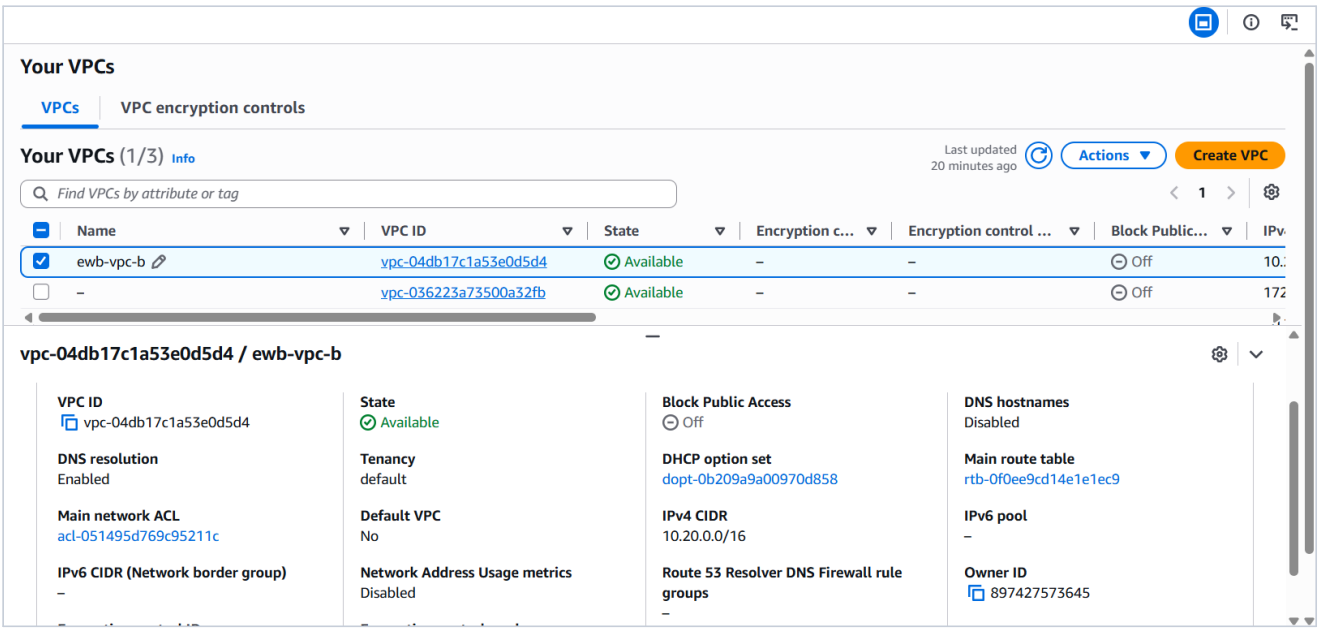


Figure 2. VPC B – ewb-vpc-b, `vpc-04db17c1a53e0d5d4`, IPv4 CIDR 10.20.0.0/16 (screenshot 161, 1:17 PM).

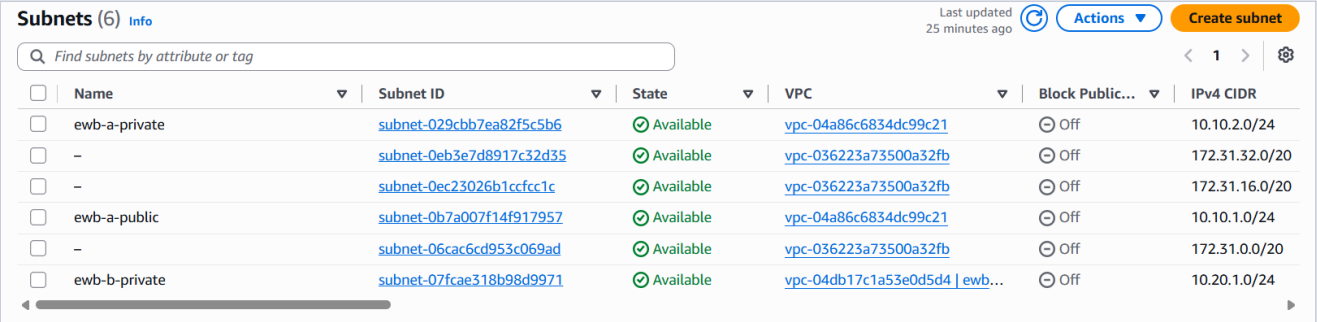


Figure 3. Subnets – ewb-a-public 10.10.1.0/24 and ewb-a-private 10.10.2.0/24 in VPC A; ewb-b-private 10.20.1.0/24 in VPC B. The other three subnets belong to the account's default VPC (172.31.0.0/16) and are not used (screenshot 163, 1:18 PM).

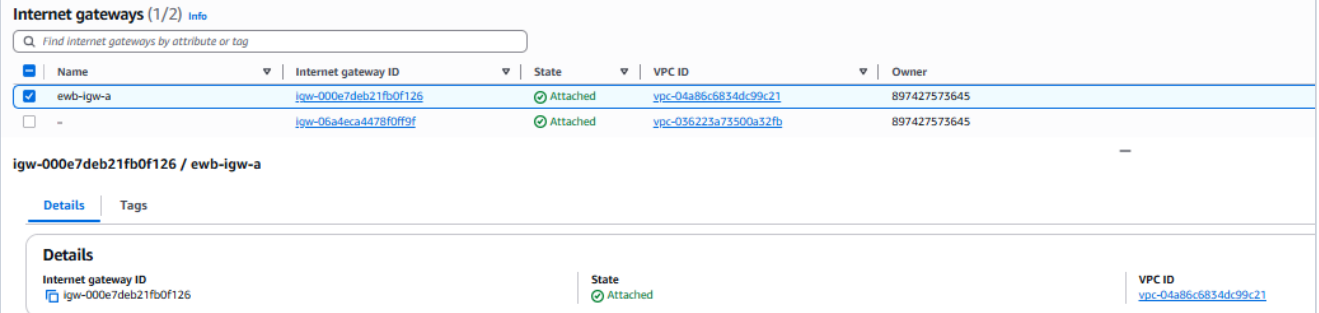


Figure 4. Internet gateway – ewb-igw-a, `igw-000e7deb21fb0f126`, state Attached to VPC A (vpc-04a86c6834dc99c21). VPC B has no internet gateway (screenshot 184, 1:48 PM).

Route tables and explicit subnet associations

Every subnet is explicitly associated with its own custom route table, so no subnet silently falls back to a main route table. *Subnets without explicit associations* is 0 in each view.

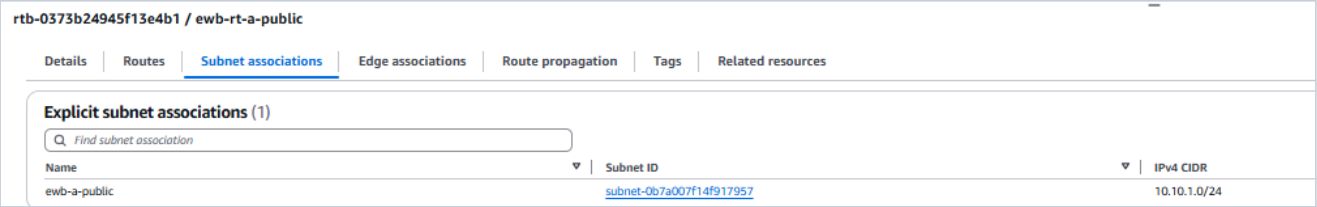


Figure 5. ewb-rt-a-public (rtb-0373b24945f13e4b1) – Subnet associations tab: ewb-a-public, 10.10.1.0/24 (screenshot 183).

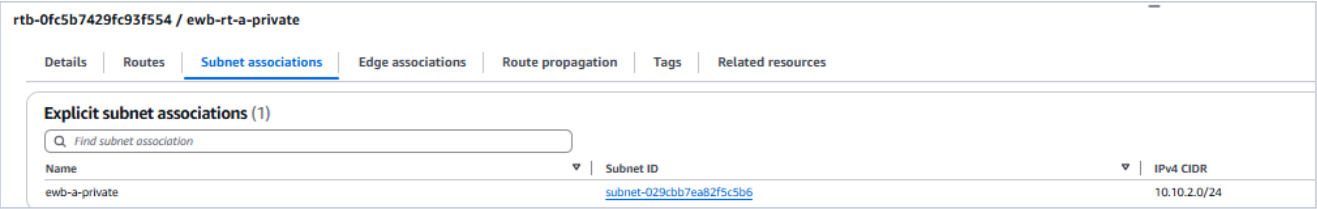


Figure 6. ewb-rt-a-private (rtb-0fc5b7429fc93f554) – Subnet associations tab: ewb-a-private, 10.10.2.0/24 (screenshot 182).

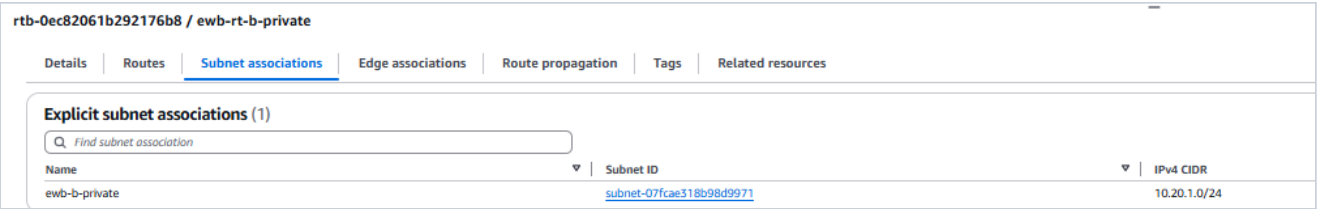


Figure 7. ewb-rt-b-private (rtb-0ec82061b292176b8) – Subnet associations tab: ewb-b-private, 10.20.1.0/24 (screenshot 181).

Final route tables (after peering)

Route table	Destination	Target	Purpose
ewb-rt-a-public	10.10.0.0/16	local	Bastion to App-A
	0.0.0.0/0	igw-000e7deb21fb0f126	Bastion internet access
	10.20.0.0/16	pcx-08b50e46f15a6073a	Bastion SSH to App-B (ProxyJump)
ewb-rt-a-private	10.10.0.0/16	local	VPC A local
	10.20.0.0/16	pcx-08b50e46f15a6073a	App-A to VPC B
ewb-rt-b-private	10.20.0.0/16	local	VPC B local
	10.10.0.0/16	pcx-08b50e46f15a6073a	Return path to VPC A

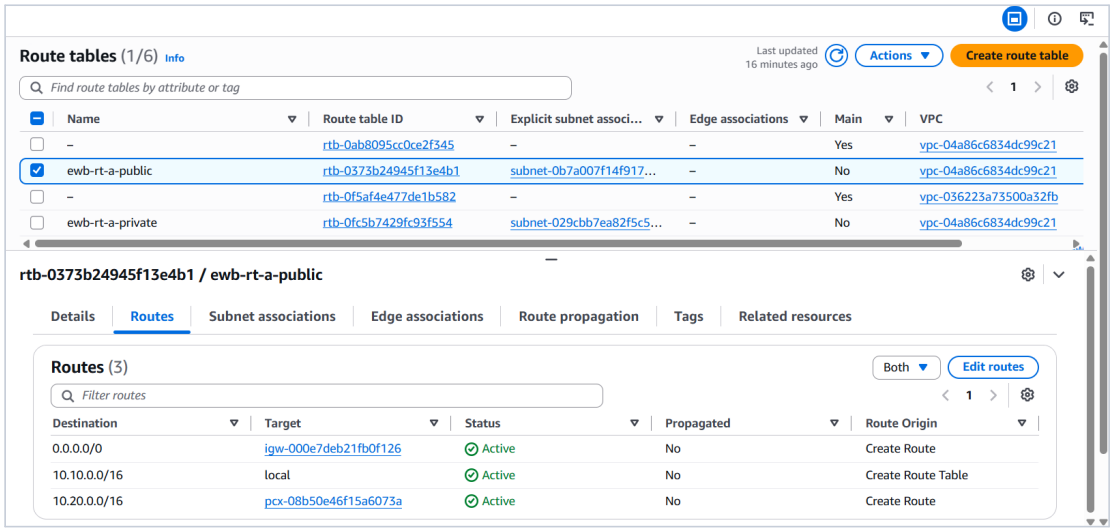


Figure 8. ewb-rt-a-public routes: `0.0.0.0/0 → igw-000e7deb21fb0f126`, `10.10.0.0/16 → local`, `10.20.0.0/16 → pcx-08b50e46f15a6073a`, all Active (screenshot 165, 1:18 PM).

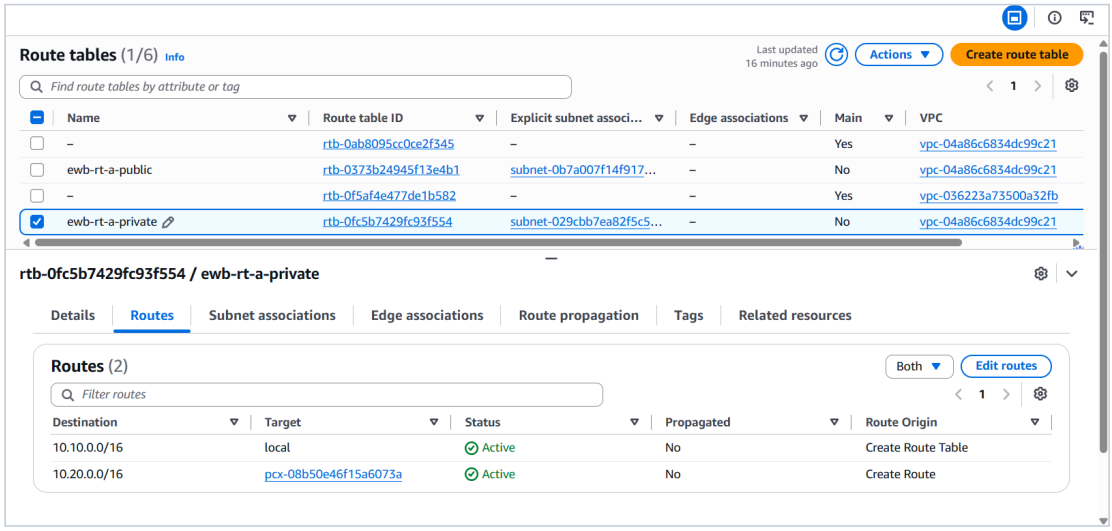


Figure 9. ewb-rt-a-private routes: `10.10.0.0/16 → local` and `10.20.0.0/16 → pcx-08b50e46f15a6073a`. No internet default route (screenshot 166).

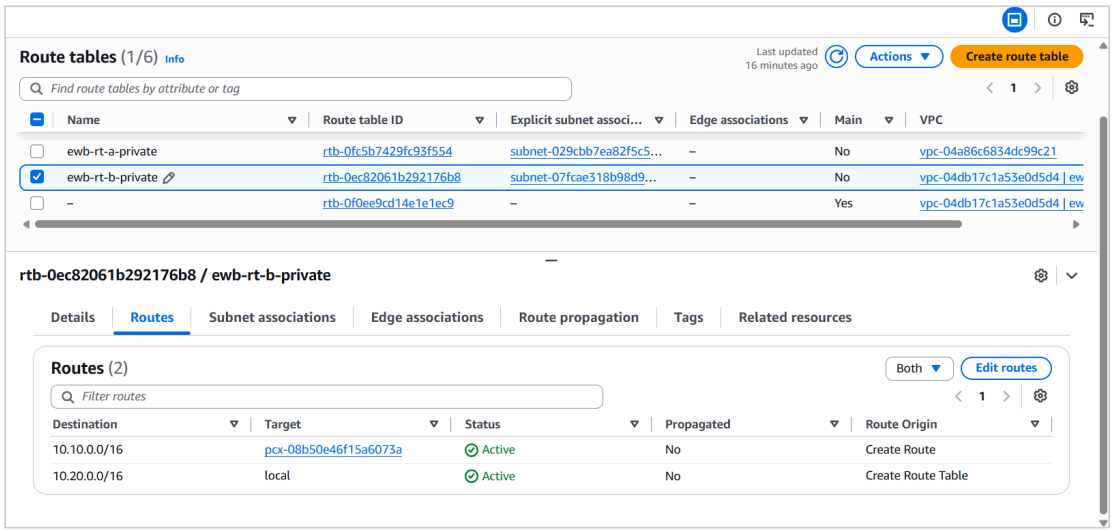


Figure 10. ewb-rt-b-private routes: `10.20.0.0/16 → local` and `10.10.0.0/16 → pcx-08b50e46f15a6073a` – the return path (screenshot 167).

04 / PEERING AND SECURITY

VPC peering connection

The peering request **ewb-a-b-peer** was created with VPC A as requester and VPC B as accepter (same account, same Region) and then **accepted** – acceptance is required even when both VPCs belong to the same account. Active peering is only the link: it carries traffic only after routes are added on both sides and the security groups allow it.

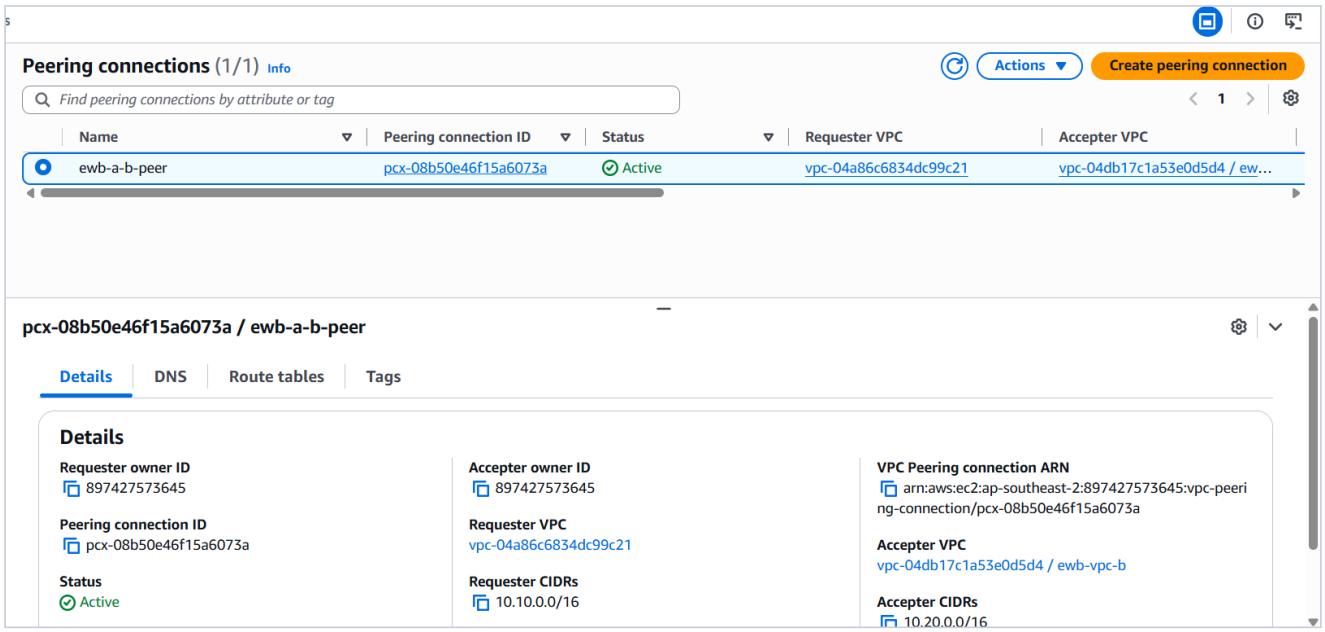


Figure 11. Peering connection – ewb-a-b-peer, `pcx-08b50e46f15a6073a`, status Active. Requester VPC vpc-04a86c6834dc99c21 (10.10.0.0/16), accepter VPC vpc-04db17c1a53e0d5d4 / ewb-vpc-b (10.20.0.0/16) (screenshot 176, 1:21 PM).

Security groups

Only the traffic the lab needs is allowed. No rule uses `0.0.0.0/0` for SSH or ICMP; outbound rules keep the default allow-all. Security groups are stateful, so replies to permitted traffic are allowed automatically.

Security group (VPC)	Type / port	Source	Why
ewb-sg-bastion (A)	SSH / TCP 22	My public IPv4 /32	Only the student's own IP can reach the bastion
ewb-sg-app-a (A)	SSH / TCP 22	10.10.1.10/32 (bastion)	Administration through the bastion only
ewb-sg-app-a (A)	Custom ICMP IPv4 – Echo Request	10.20.1.10/32 (App-B)	Lets App-B ping App-A (reverse test)
ewb-sg-app-b (B)	SSH / TCP 22	10.10.1.10/32 (bastion)	App-B sees the bastion's private IP via ProxyJump
ewb-sg-app-b (B)	Custom ICMP IPv4 – Echo Request	10.10.2.10/32 (App-A)	Lets App-A ping App-B (forward test)

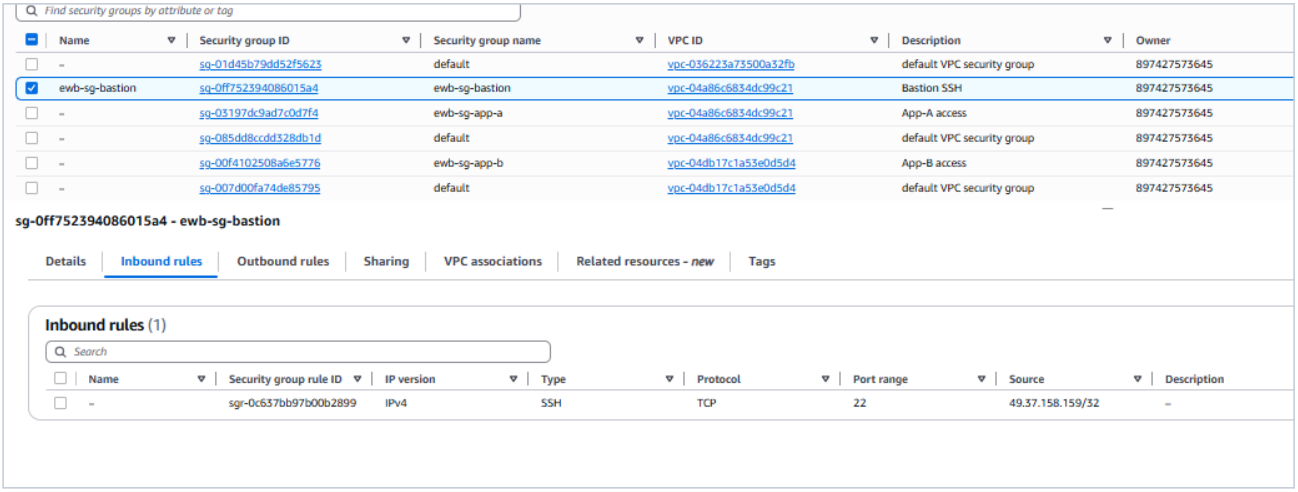


Figure 12. ewb-sg-bastion (sg-0ff752394086015a4, VPC A) – one inbound rule: SSH, TCP 22, source = the student's current public IPv4 /32 (screenshot 178, 1:45 PM). The other three groups in the list are AWS default groups.

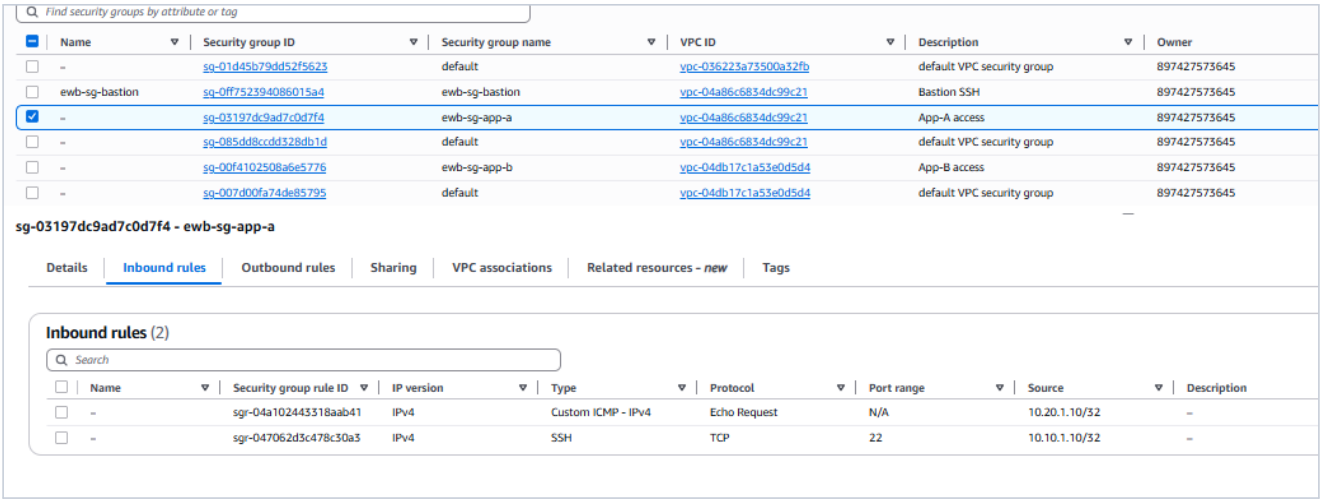


Figure 13. ewb-sg-app-a (sg-03197dc9ad7c0d7f4, VPC A) – Custom ICMP IPv4 Echo Request from 10.20.1.10/32 and SSH TCP 22 from 10.10.1.10/32 (screenshot 179, 1:46 PM).

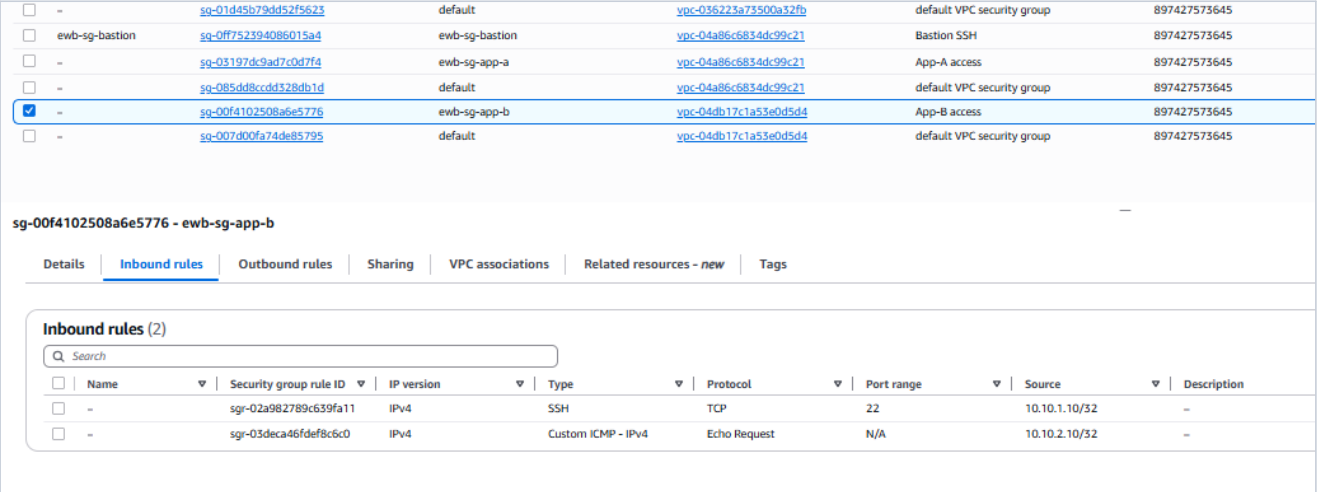


Figure 14. ewb-sg-app-b (sg-00f4102508a6e5776, VPC B) – SSH TCP 22 from 10.10.1.10/32 and Custom ICMP IPv4 Echo Request from 10.10.2.10/32 (screenshot 180, 1:46 PM).

05 / COMPUTE

EC2 instances and isolation (Test 05)

Three instances share one key pair (`ewb-lab`), AMI and instance type. The bastion is the only instance with a public IPv4 address; App-A and App-B show an empty Public IPv4 field and therefore cannot be reached from the internet.

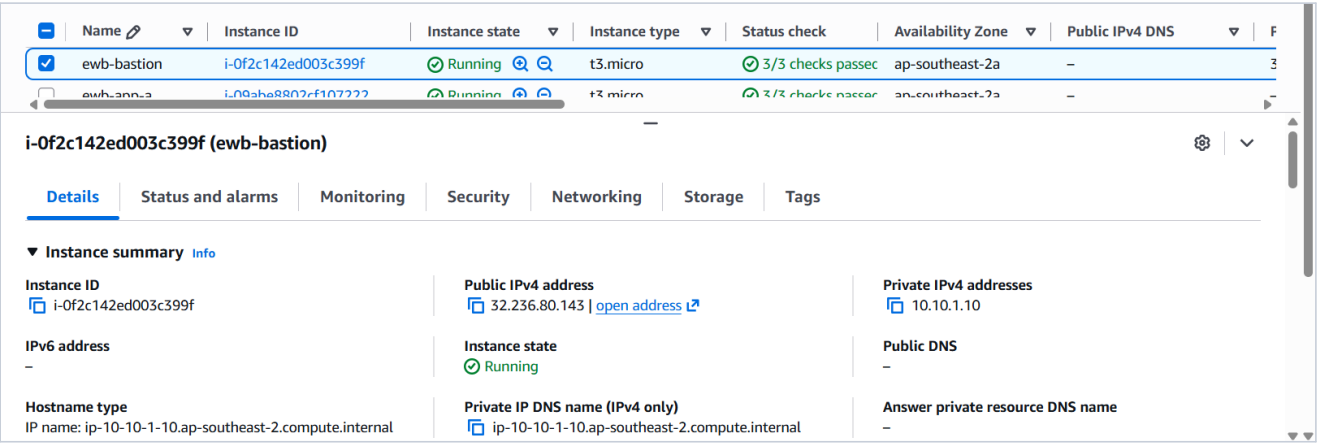


Figure 15. ewb-bastion – i-0f2c142ed003c399f, Running, public IPv4 32.236.80.143, private IPv4 10.10.1.10 (screenshot 173, 1:20 PM).

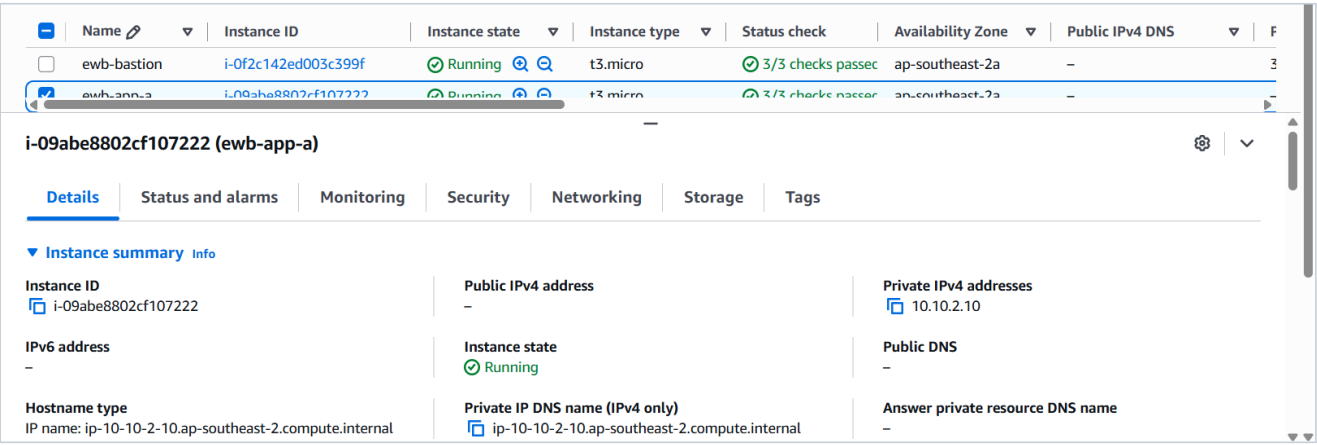


Figure 16. Test 05 – ewb-app-a – i-09abe8802cf107222, Running, **Public IPv4 address: –**, private IPv4 10.10.2.10 (screenshot 174, 1:20 PM).

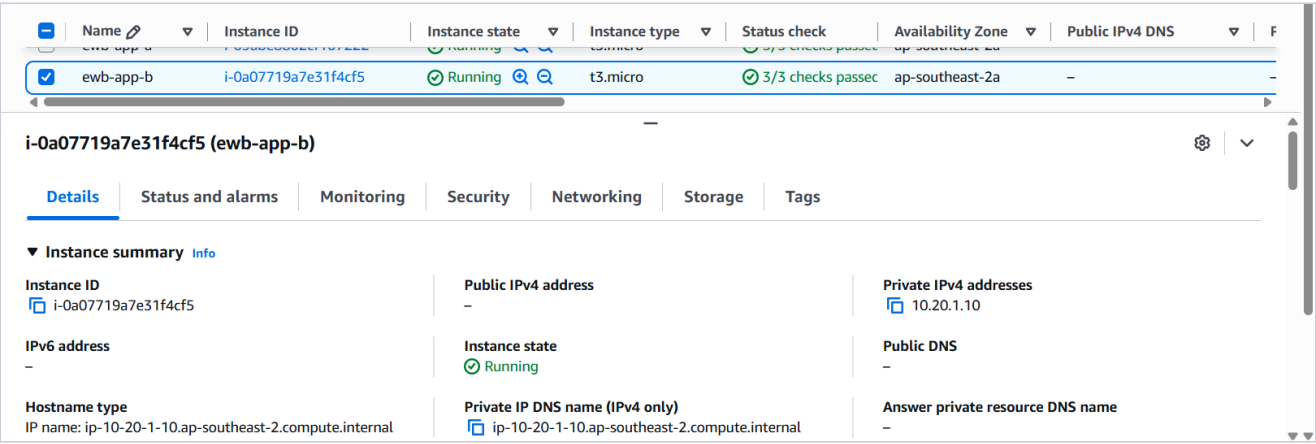


Figure 17. Test 05 – ewb-app-b – i-0a07719a7e31f4cf5, Running, **Public IPv4 address: –**, private IPv4 10.20.1.10 (screenshot 175, 1:20 PM).

06 / VERIFICATION

Positive and negative tests

SSH access uses ProxyJump from the laptop (Windows PowerShell with the built-in OpenSSH client). The private key stays on the laptop and is never copied to an instance. The bastion's private prompt (`ip-10-10-1-10`), App-A's (`ip-10-10-2-10`) and App-B's (`ip-10-20-1-10`) confirm the fixed private addresses.

Test 01 – baseline, before peering (expected: no replies)

```
ec2-user@ip-10-10-2-10:~
Warning: Permanently added '32.236.80.143' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-10-1-10 ~]$ exit
logout
Connection to 32.236.80.143 closed.
PS C:\WINDOWS\system32> ssh ewb-app-a
The authenticity of host '10.10.2.10 (<no hostip for proxy command>)' can't be established.
ED25519 key fingerprint is SHA256:66OphnC5NmeE24s0W109OYtJbsbYZk8h3+FcXPqWtLY.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.2.10' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-10-2-10 ~]$ hostname
hostname -I
ping -c 4 10.20.1.10
ip-10-10-2-10.ap-southeast-2.compute.internal
10.10.2.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3121ms

[ec2-user@ip-10-10-2-10 ~]$
```

Figure 18. Host: ewb-app-a (10.10.2.10). Test 01. Before any peering or peer routes, `ping -c 4 10.20.1.10` shows 4 packets transmitted, 0 received, **100% packet loss**. The same terminal first shows SSH to the bastion and to App-A working through VPC A's local route (screenshot 147, 12:52 PM).

Test 02 – forward path, after peering and routes

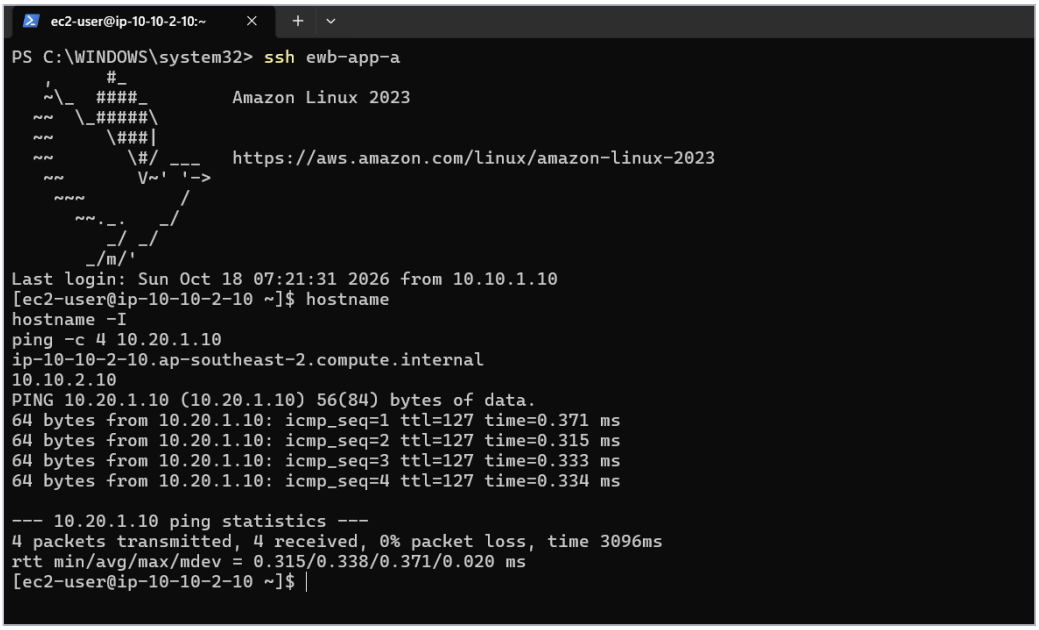


Figure 19. Host: ewb-app-a (10.10.2.10). Test 02. `ping -c 4 10.20.1.10` → 4 transmitted, 4 received, **0% packet loss**, avg 0.338 ms (screenshot 151, 1:05 PM).

Tests 03 and 04 – reverse path and private SSH to App-B through the bastion

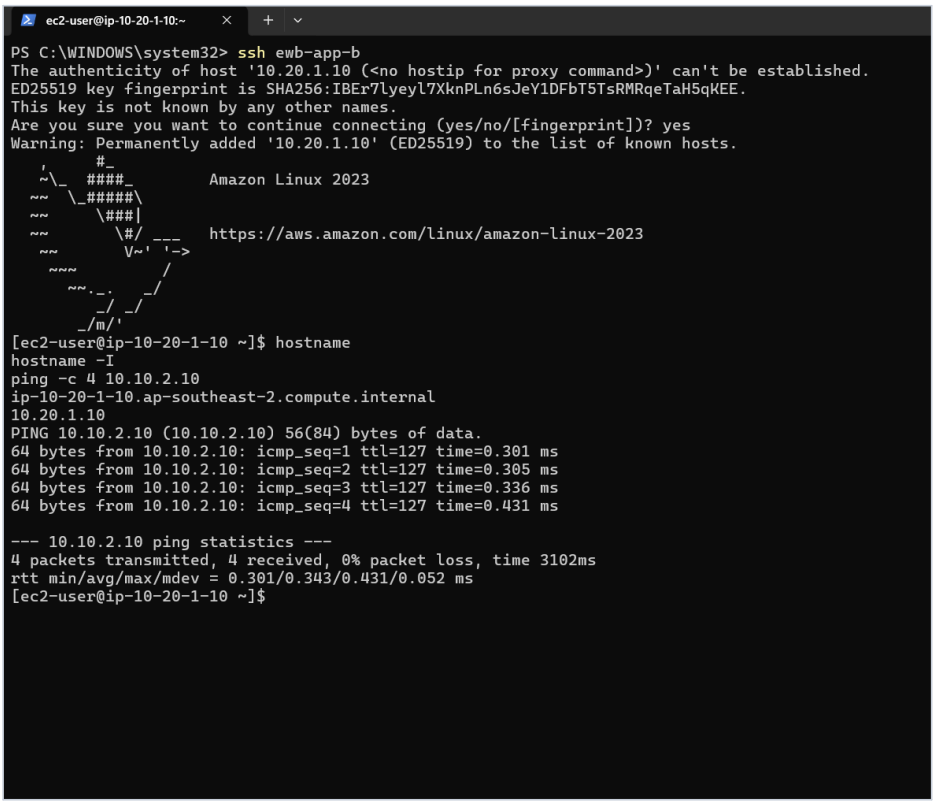


Figure 20. Host: ewb-app-b (10.20.1.10). Tests 03 and 04. `ssh ewb-app-b` from the laptop opens a shell on `ip-10-20-1-10` through the bastion (the first-connection host-key prompt is accepted after the route exists). From App-B, `ping -c 4 10.10.2.10` → 4 received, **0% packet loss** (screenshot 152, 1:06 PM).

07 / CONTROLLED FAILURE

Controlled ICMP failure and recovery

To prove that the security group – not the route or the peering – controls this traffic, only the **Custom ICMP Echo Request** rule on `ewb-sg-app-b` (source 10.10.2.10/32) was removed; the SSH rule stayed in place. The forward ping from App-A then failed. The exact rule was restored and the ping recovered. The test was performed twice (run 1 at about 1:11–1:15 PM, run 2 at about 1:58–1:59 PM).

Run 1 – complete failure and recovery cycle

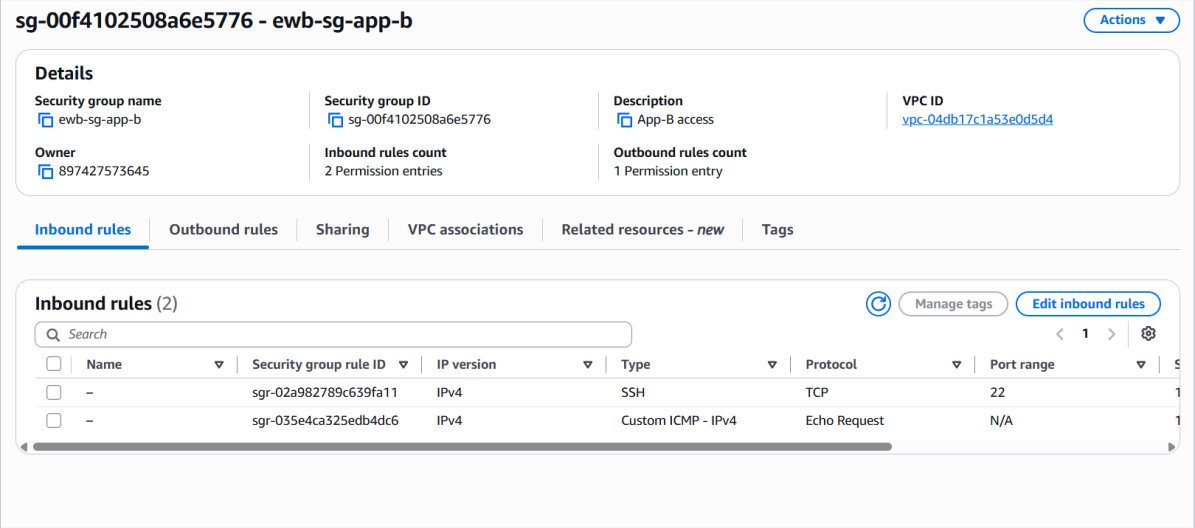


Figure 21. Before. ewb-sg-app-b has two inbound rules: SSH (sgr-02a982789c639fa11) and Custom ICMP IPv4 Echo Request (sgr-035e4ca325edb4dc6) (screenshot 156, 1:11 PM).

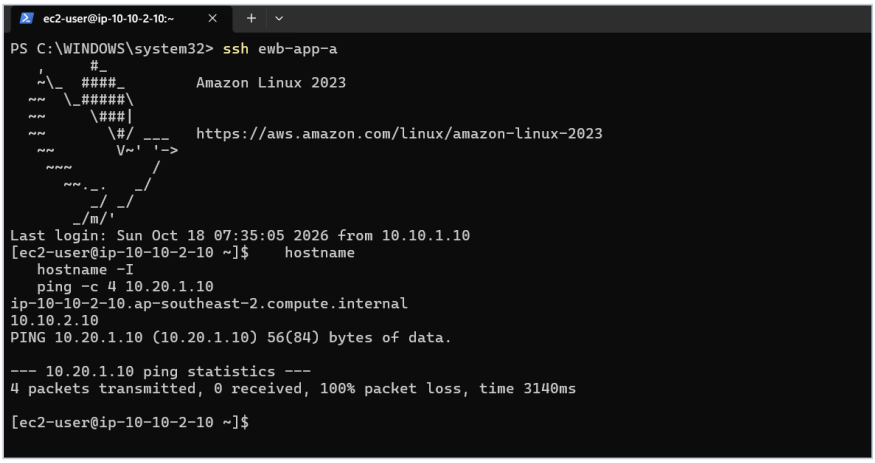


Figure 22. Host: ewb-app-a. Failure. With the Echo Request rule removed, `ping -c 4 10.20.1.10` → 4 transmitted, 0 received, **100% packet loss**. Routes and peering are unchanged (screenshot 157, 1:13 PM).

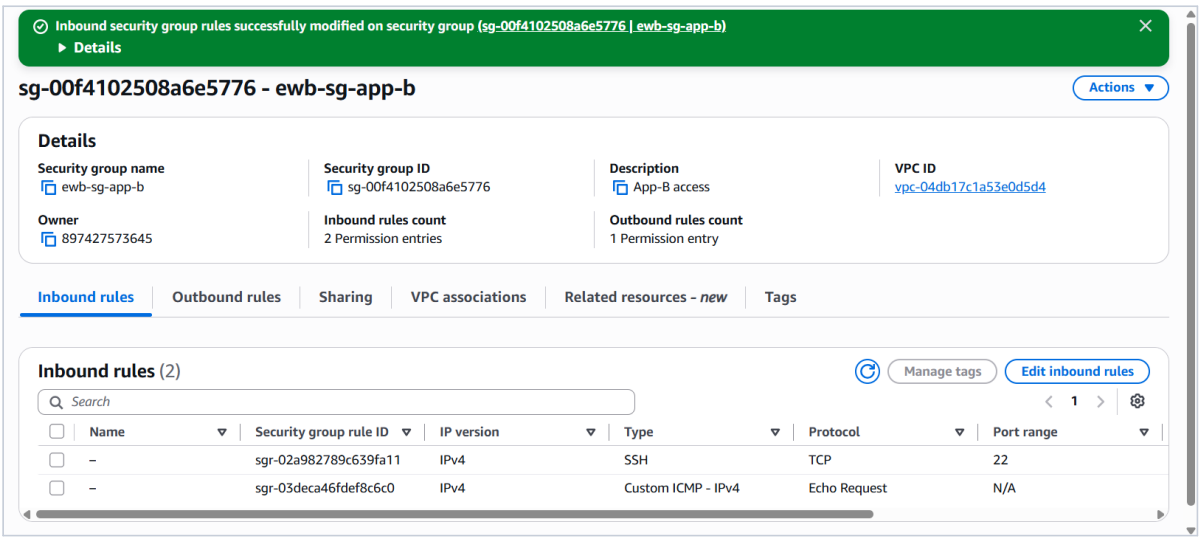


Figure 23. Restored. The same rule was re-added (Custom ICMP IPv4, Echo Request, source 10.10.2.10/32). The new rule ID `sgr-03deca46fdef8c6c0` replaces the deleted `sgr-035e4ca325edb4dc6`, which shows the rule really was removed and re-created (screenshot 158, 1:14 PM).

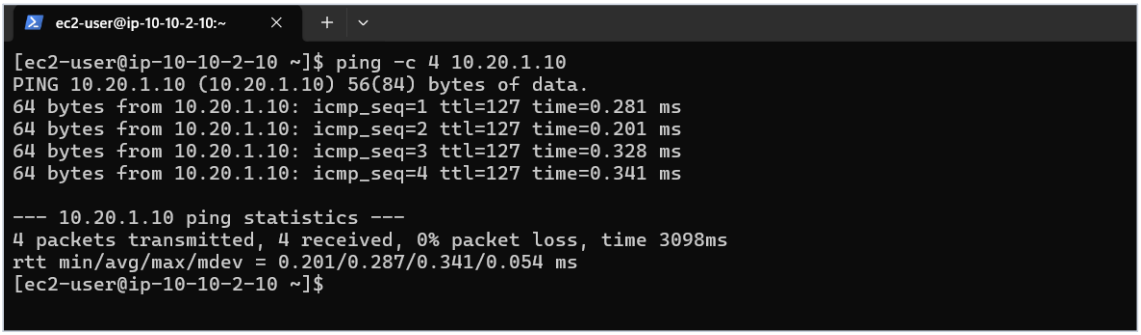


Figure 24. Host: ewb-app-a (10.10.2.10). Recovery. `ping -c 4 10.20.1.10` → 4 transmitted, 4 received, **0% packet loss** (screenshot 159, 1:15 PM).

Run 2 – repeat of the failure

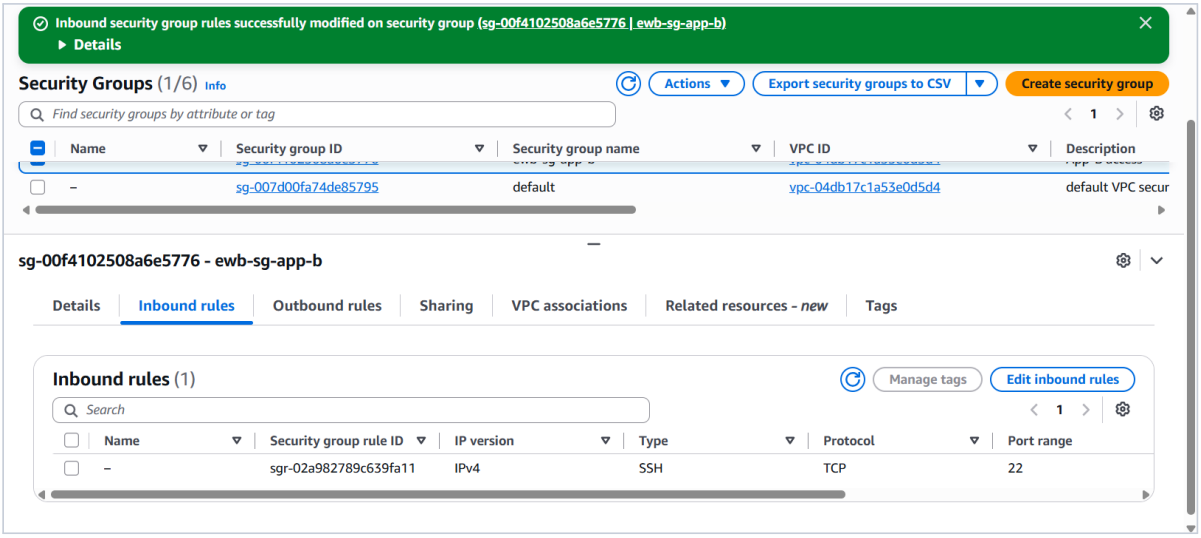


Figure 25. Rule removed. Console confirms *Inbound security group rules successfully modified* on ewb-sg-app-b; Inbound rules (1) now lists only SSH, TCP 22 (screenshot 187, 1:58 PM).

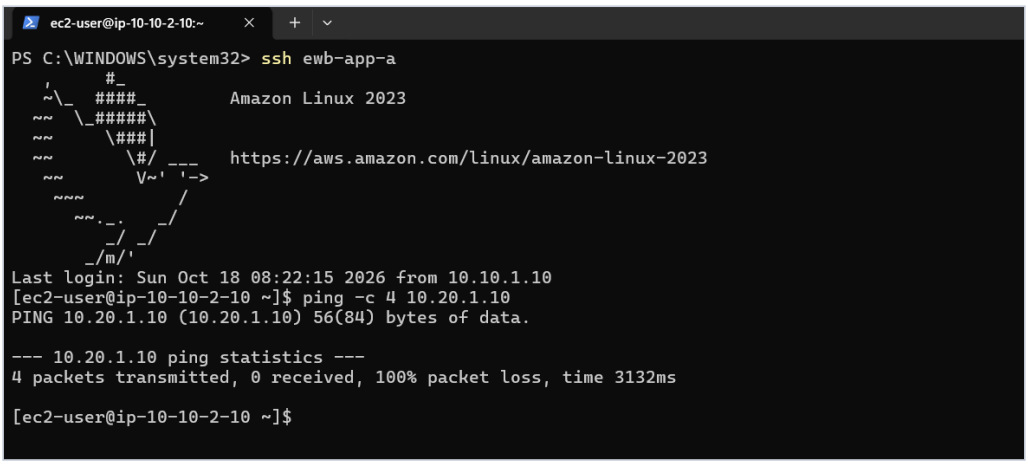


Figure 26. Host: ewb-app-a (ip-10-10-2-10). Failure, run 2. `ping -c 4 10.20.1.10` → 0 received, **100% packet loss** while only the SSH rule exists (screenshot 188, 1:59 PM).

What this proves

Active peering and correct routes are necessary but not sufficient. A destination security group that does not allow the traffic still drops it, and the failure disappears as soon as the exact rule is restored.

Additional test – the route is required too

As a second negative test, the peering route `10.20.0.0/16 → pcx-08b50e46f15a6073a` was deleted from `ewb-rt-a-private`, leaving only the local route. With the peering still Active and the security groups unchanged, App-A could no longer reach App-B.

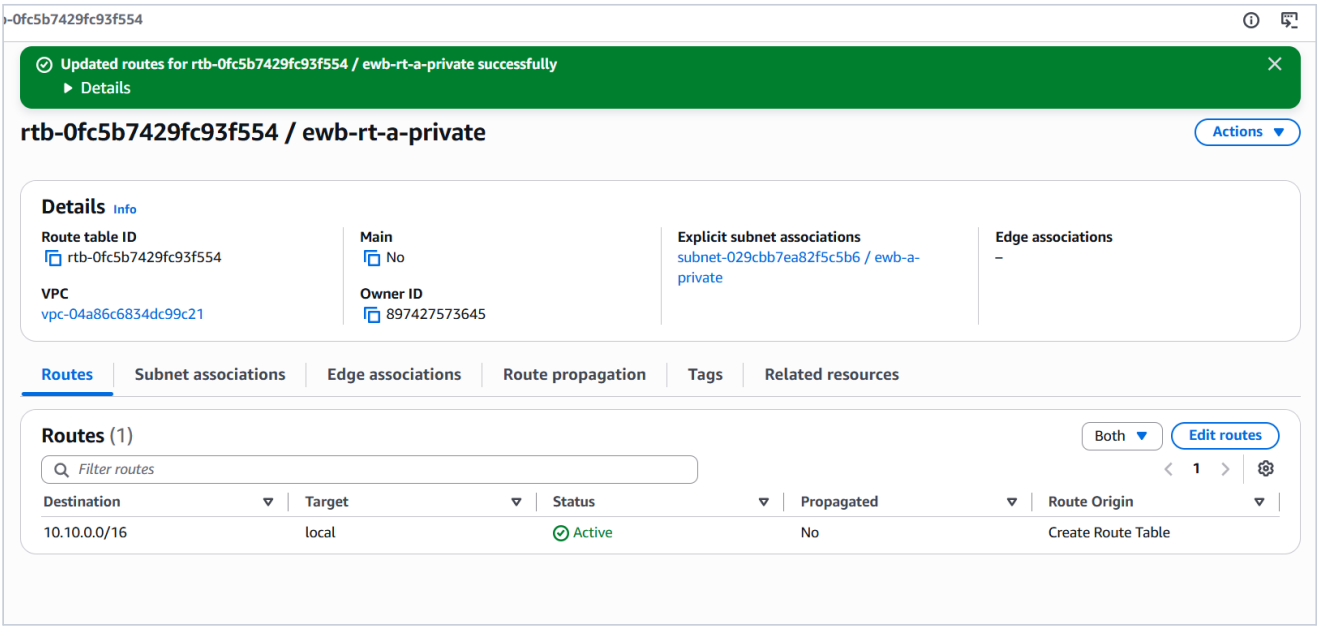


Figure 27. Peer route removed. ewb-rt-a-private now shows Routes (1): only `10.10.0.0/16 → local` (screenshot 189, 2:01 PM).

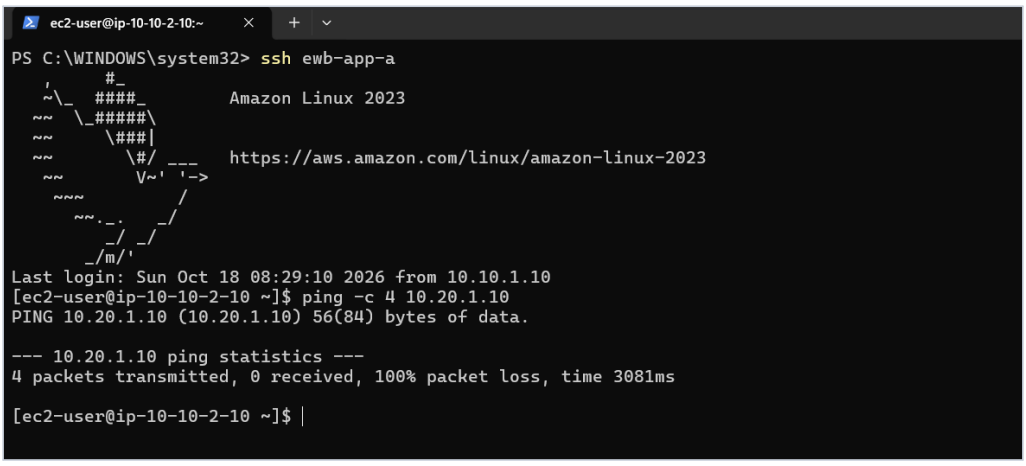


Figure 28. Host: ewb-app-a (ip-10-10-2-10). `ping -c 4 10.20.1.10` → 4 transmitted, 0 received, **100% packet loss** without the peer route (screenshot 190, 2:02 PM).

08 / FORWARD AND RETURN PATH

How the traffic flows

Forward path (App-A → App-B). App-A (10.10.2.10) sends an ICMP Echo Request to 10.20.1.10. The subnet's route table *ewb-rt-a-private* has no local match, but its `10.20.0.0/16 → pcx-08b50e46f15a6073a` route does, so the packet is sent across the peering connection into VPC B. *ewb-sg-app-b* allows Echo Request from 10.10.2.10/32, so App-B accepts it.

Return path (App-B → App-A). App-B replies to 10.10.2.10. *ewb-rt-b-private* sends `10.10.0.0/16` to the same peering connection, so the reply returns to VPC A. The security group is stateful, so the reply needs no inbound rule – but it does need this route. A one-way route is not enough.

Administration path. Laptop → bastion (public IPv4, SSH from the student's /32 only) → App-A or App-B (SSH only from 10.10.1.10/32). For App-B, ProxyJump makes the bastion open a new connection, so the bastion's subnet (*ewb-rt-a-public*) needs the `10.20.0.0/16` peering route; that more specific route is chosen over the internet default route.

Viva questions

Question	Answer
Why is the return route needed?	A reply is a new packet addressed to the other VPC. Without a route to that CIDR via the peering connection it has no path back and is dropped, even though the security group allows the reply.
Why is App-B's SSH source the bastion's private IP?	ProxyJump makes the bastion start its own SSH connection to App-B. App-B therefore sees the bastion's private address 10.10.1.10 as the source, not the laptop's public IP.
Why does Active peering not guarantee a successful ping?	Peering only creates the link. Traffic also needs routes on both sides and a security group rule on the destination. Removing either one (see Figures 22 and 28) breaks the ping although peering stays Active.
Can VPC B use VPC A's internet gateway over peering?	No. VPC peering is one-to-one and non-transitive; an internet gateway cannot be shared through a peering connection. VPC B has no internet access, which is intended.

09 / CLEANUP AND REFLECTION

Cleanup

Evidence was captured first; only the resources created for this project are then removed, in dependency order, and the account is verified clear using the tag `Project = EWB-VPC-Peering`.

Step	Action	Status
1	Terminate ewb-bastion, ewb-app-a, ewb-app-b; confirm the lab EBS volumes are deleted; delete key pair ewb-lab and the local .pem	Pending
2	Delete the peer routes and the peering connection ewb-a-b-peer	Pending
3	Delete security groups ewb-sg-bastion, ewb-sg-app-a, ewb-sg-app-b	Pending
4	Delete the three subnets and the three custom route tables	Pending
5	Detach and delete ewb-igw-a	Pending
6	Delete VPC A and VPC B	Pending
7	Filter by the project tag; check EC2, volumes, public IPv4 allocations, VPC resources and Billing	Pending

Cleanup evidence

Console confirmations captured during cleanup on 18 October 2026, 5:03–5:07 PM. Figures 29–35 follow the order of the steps in the table above.

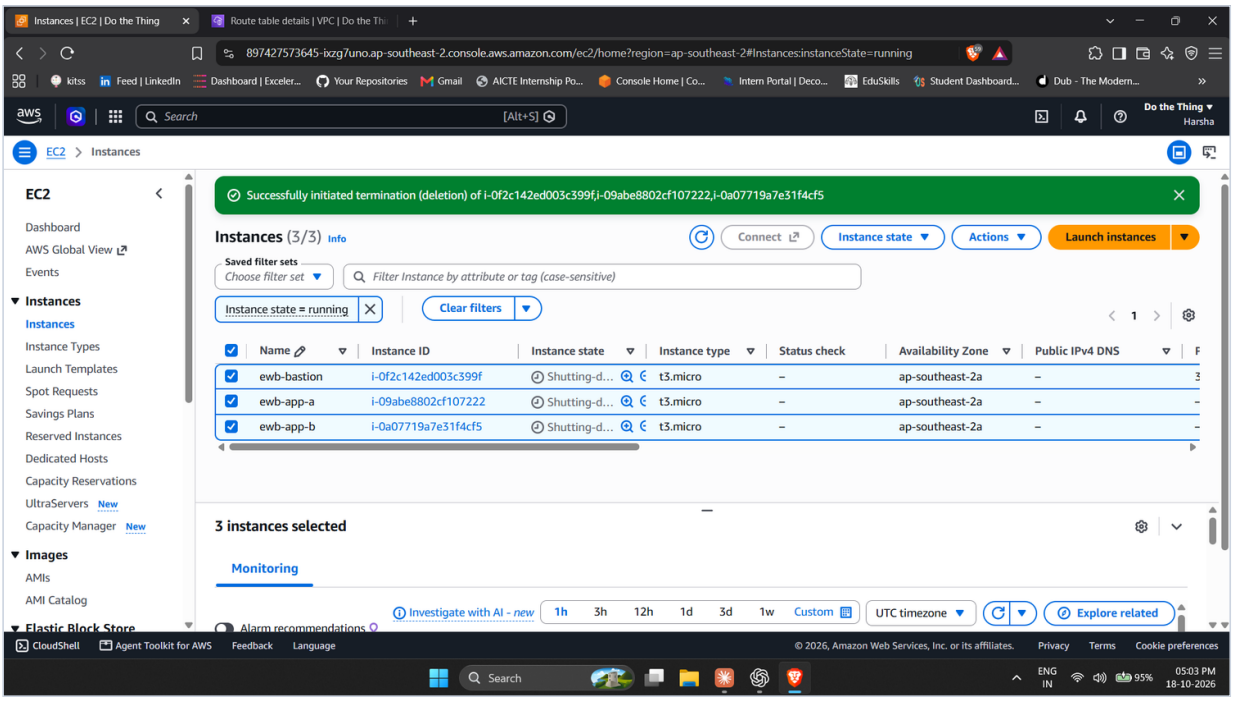


Figure 29. Step 1 – instances terminating. Console message “Successfully initiated termination (deletion)” for `i-0f2c142ed003c399f` (ewb-bastion), `i-09abe8802cf107222` (ewb-app-a) and `i-0a07719a7e31f4cf5` (ewb-app-b); all three show Shutting-down. (screenshot 191, 5:03 PM).

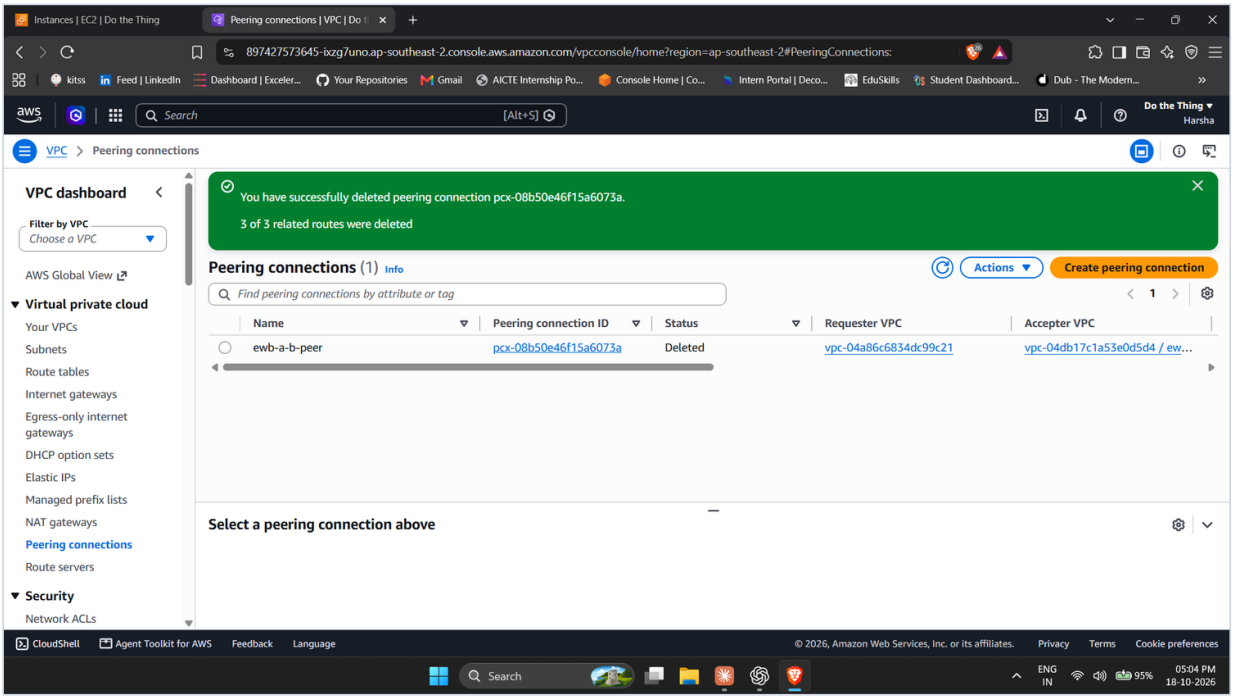


Figure 30. Step 2 – peering connection deleted. `pcx-08b50e46f15a6073a` (ewb-a-b-peer) has status Deleted; the console reports “3 of 3 related routes were deleted”. (screenshot 192, 5:04 PM).

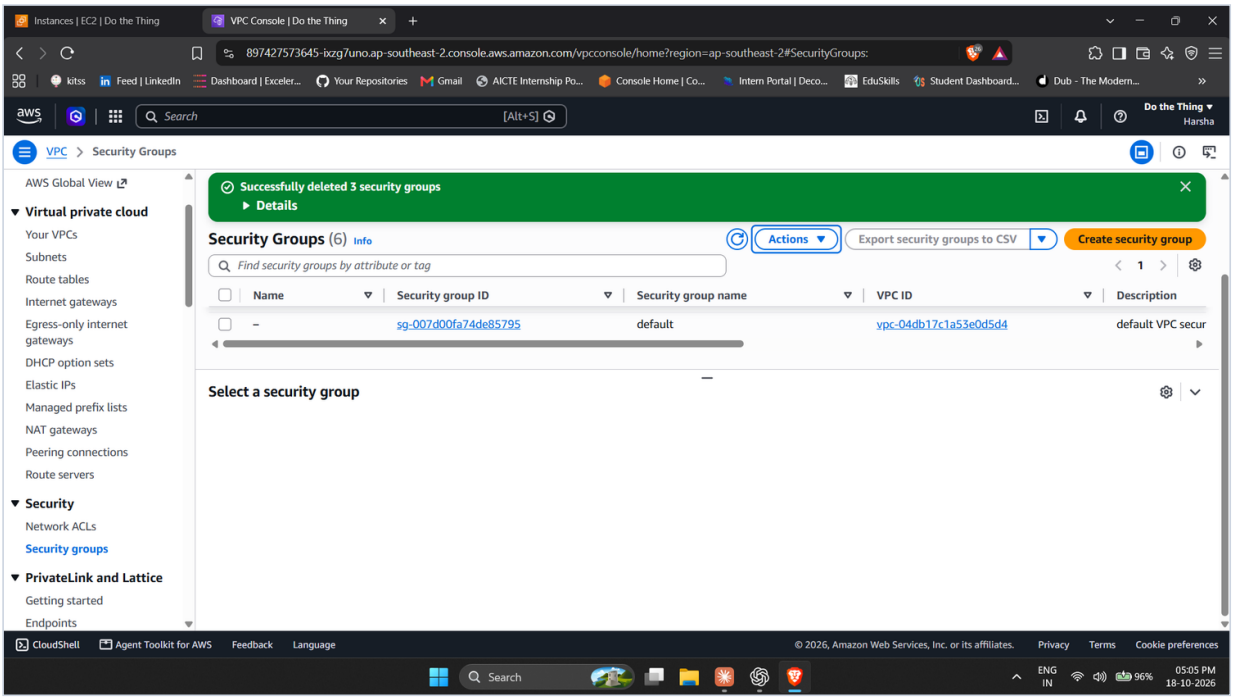


Figure 31. Step 3 – security groups deleted. Console message “Successfully deleted 3 security groups”. Only an AWS default group (`sg-007d00fa74de85795`) remains in the list. (screenshot 193, 5:05 PM).

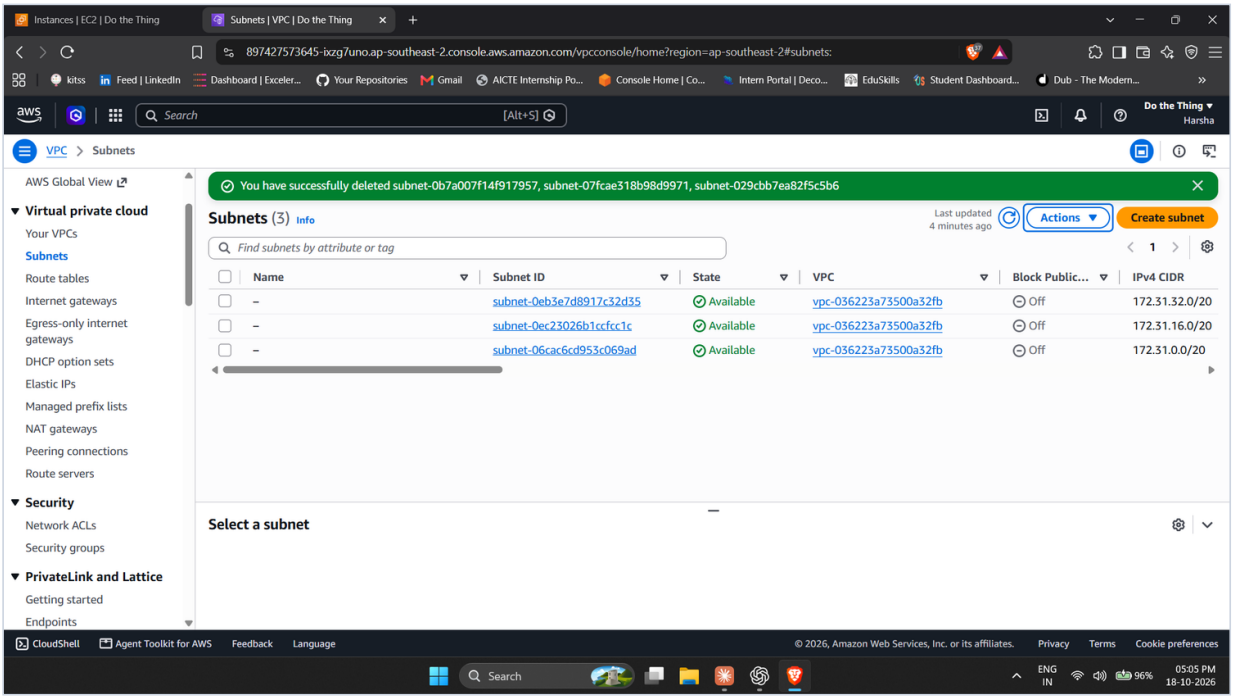


Figure 32. Step 4 – subnets deleted. `subnet-0b7a007f14f917957`, `subnet-07fcae318b98d9971` and `subnet-029cbb7ea82f5c5b6` deleted. The three subnets left belong to the account's default VPC (172.31.0.0/16). (screenshot 194, 5:05 PM).

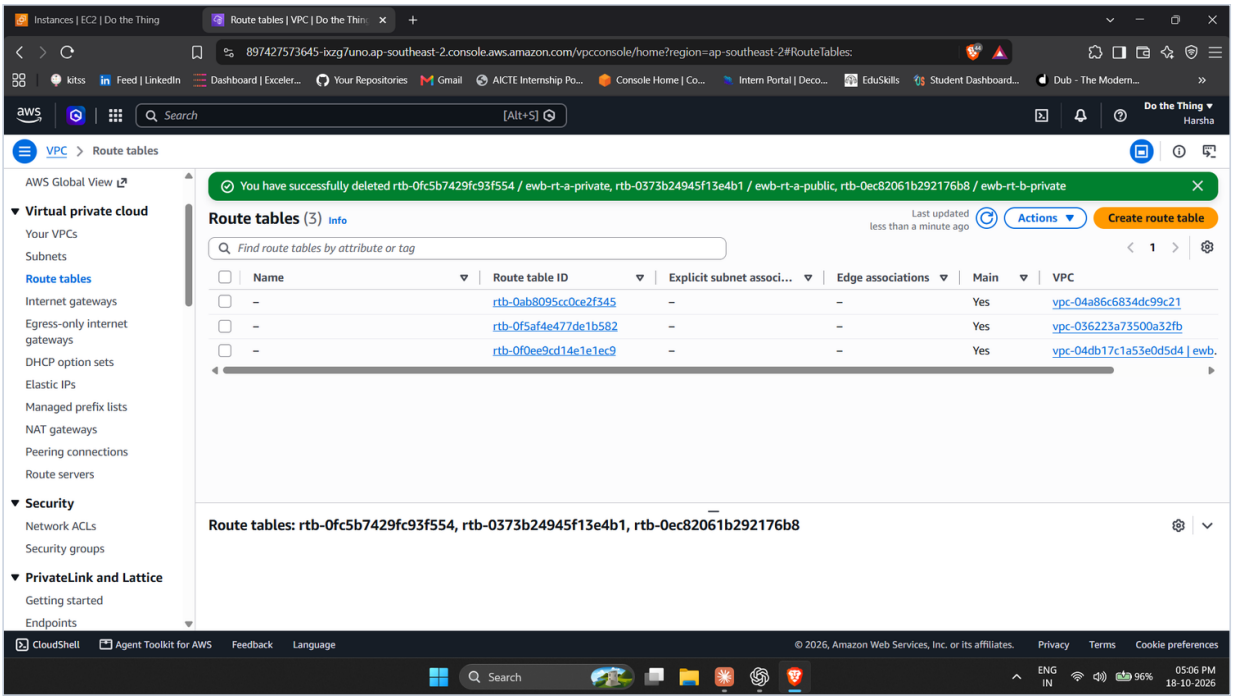


Figure 33. Step 4 – custom route tables deleted. ewb-rt-a-private, ewb-rt-a-public and ewb-rt-b-private deleted. The three route tables left are main route tables (Main = Yes). (screenshot 195, 5:06 PM).

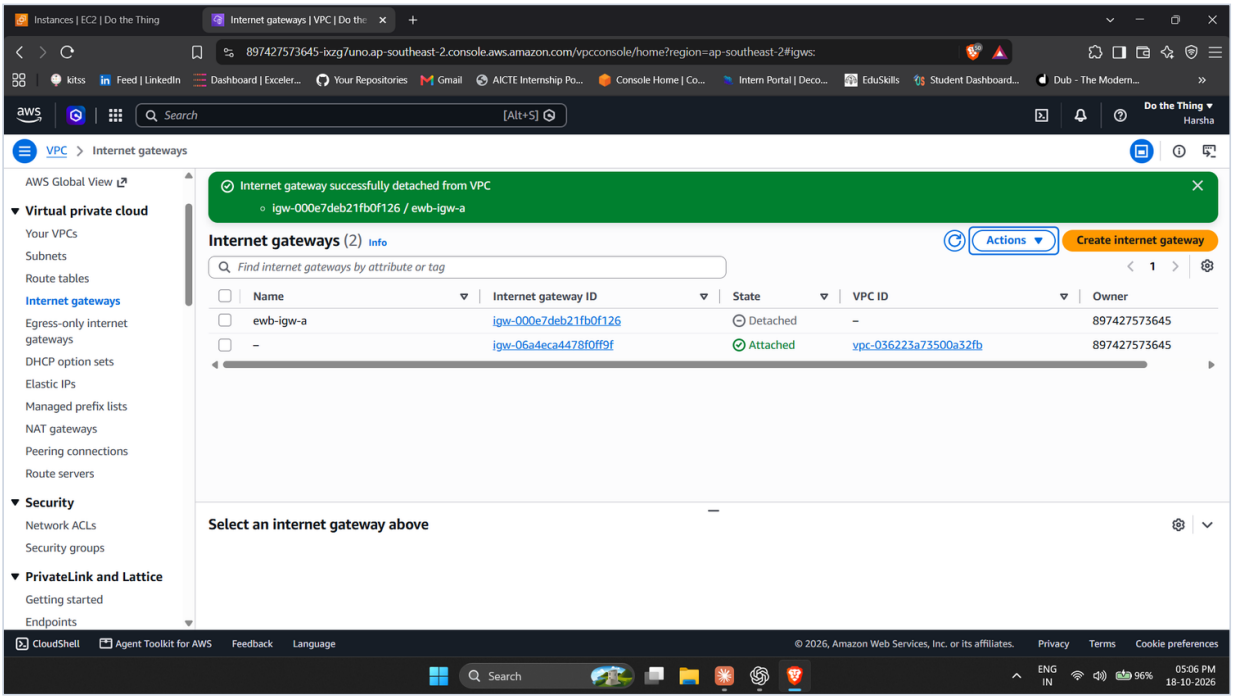


Figure 34. Step 5 – internet gateway detached. "Internet gateway successfully detached from VPC" for `igw-000e7deb21fb0f126` (ewb-igw-a); state is Detached with no VPC. (screenshot 196, 5:06 PM).

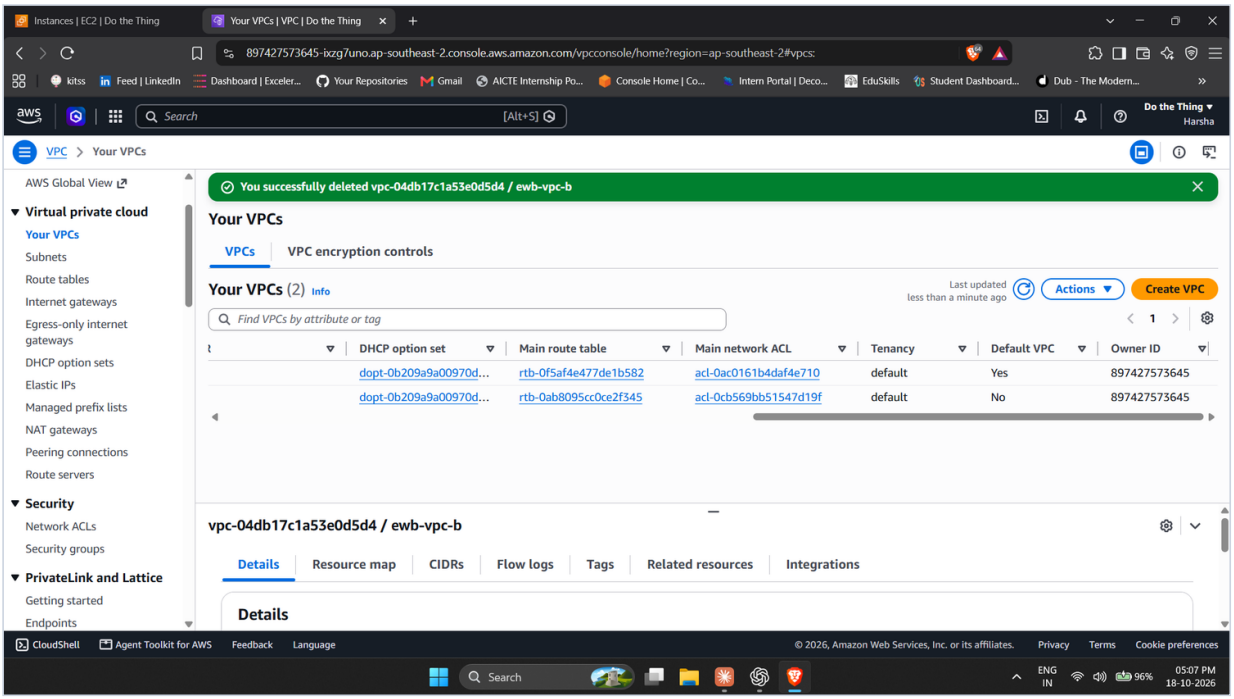


Figure 35. Step 6 – VPC B deleted. Console message "You successfully deleted `vpc-04db17c1a53e0d5d4 / ewb-vpc-b`". Your VPCs now lists two VPCs. (screenshot 197, 5:07 PM).

What I learned

- **Design first.** Non-overlapping VPC CIDRs are a hard prerequisite for peering; planning the address ranges and fixed private IPs up front made every /32 security rule predictable.
- **Three things must line up.** A working peering path needs the Active connection, routes in *both* directions, and a security group rule on the destination. I proved each one by removing it and watching the ping fail, then restoring it.
- **Least privilege.** Using only SSH from specific /32 sources and only ICMP Echo Request (not “All ICMP”), and keeping App-A and App-B without public IPs, keeps the attack surface to a single bastion. I caught and fixed an “All ICMP” rule on ewb-sg-app-a during the build.
- **Bastion and ProxyJump.** The private key never leaves the laptop; the bastion only relays the connection, which is why the destination sees the bastion's private IP.
- **Cost and hygiene.** The lab needs no NAT gateway, resources are tagged for filtering, and cleanup is done in dependency order so nothing keeps billing.